



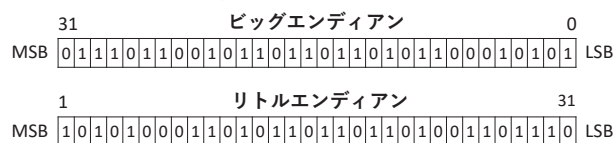
## エンディアン 由来は「ガリバー旅行記」から

- コンピュータの中でどのようにデータが格納されているか

- バイトオーダー 重要

| ビッグエンディアン | 00 | 01 | 02 | 03 |
|-----------|----|----|----|----|
| リトルエンディアン | 03 | 02 | 01 | 00 |

- ビットオーダー 通常あまり気にすることはない



## エンディアン

- プロセッサ

| プロセッサ             | エンディアン       |
|-------------------|--------------|
| インテル/AMD x86(x64) | リトル          |
| PowerPC           | ビッグ          |
| SPARC             | ビッグ          |
| MIPS              | 両対応 (ビッグが多い) |
| ARM               | 両対応          |

- データ

| 種類              | エンディアン |
|-----------------|--------|
| BMP (画像データ)     | リトル    |
| USB             | リトル    |
| TCP/IP (ネットワーク) | ビッグ    |

## 2の補数表現(1/3)

- 最上位ビットを負数と考える

- 符号なしの5

|   |   |   |   |   |
|---|---|---|---|---|
| 5 | 8 | 4 | 2 | 1 |
|   | 0 | 1 | 0 | 1 |

- 符号ありの-5

|    |   |   |   |   |
|----|---|---|---|---|
| -5 | 8 | 4 | 2 | 1 |
|    | 1 | 0 | 1 | 1 |

$$\Rightarrow -8_{10} + 2_{10} + 1_{10} = -5_{10}$$

## 2の補数表現(3/3)

- 符号あり/なし10進数との対応

| 2進数 | 符号付き10進数 | 符号なし10進数 |
|-----|----------|----------|
| 000 | 0        | 0        |
| 001 | 1        | 1        |
| 010 | 2        | 2        |
| 011 | 3        | 3        |
| 100 | -4       | 4        |
| 101 | -3       | 5        |
| 110 | -2       | 6        |
| 111 | -1       | 7        |

大小関係进行比较するときは符号の有無に注意！

## メモリやHDDの中のデータ表現

| アドレス | データ |
|------|-----|
| 0    | 12  |
| 1    | 34  |
| 2    | 38  |
| 3    | 5F  |
| 4    | 1D  |
| 5    | F8  |
| 6    | DC  |

- 16bit (2byte) 変数:x
- メモリの0番地から2byte読み出す
- 読みだした値xは
  - 0x1234?
  - 0x3412?
- 前者がビッグエンディアン、後者がリトルエンディアン
- 異種間でデータ交換 (ストレージの読み書き、通信) する際に問題になる！

## 符号付き数と符号なし数

- 符号なし数

- 0011<sub>2</sub> ⇒ 3<sub>10</sub>
- 1010<sub>2</sub> ⇒ 10<sub>10</sub>

- 符号付き数

- どうやって表現する？
- 符号ビットを付加
  - 先頭に正なら0、負なら1を付加
  - +3 ⇒ +0010<sub>2</sub> ⇒ 0010<sub>2</sub>
  - 3 ⇒ -0010<sub>2</sub> ⇒ 10010<sub>2</sub>
  - ⇒ 0は00000? 10000?

## 2の補数表現(2/3)

- 正の数は先頭に0を付加

- 負の数は先頭に1を付加し、各ビットを反転させてから1を加える

- 例) +3 ⇒ -3に変換



## 値域

- 同じビット数でも符号のありか符号なしかで扱える数の範囲が異なる

| ビット数 | 符号あり                            | 符号なし              |
|------|---------------------------------|-------------------|
| 8    | -128 ~ +127                     | 0 ~ 255           |
| 16   | -32,768 ~ +32,767               | 0 ~ 65,535        |
| 32   | -2,147,483,648 ~ +2,147,483,647 | 0 ~ 4,294,967,295 |

- C言語のchar, short, longの値域とは異なるので注意！

## 例題

- 例題1
  - -19を2の補数表現で表せ。ただし、語長は8bitとする。
- 例題2
  - 2の補数表現で表された次の数値を10進数で表せ。
    - 00101100
    - 10101100

## 例題2

- 問題
  - 2の補数表現で表された次の数値を10進数で表せ。
    - 00101100
    - 10101100
- 解答

## 2進数と16進数

- コンピュータの内部は基本的に2進数
    - 1010110100101010011101011010101
  - 桁が多くなるため、人間が扱いにくい
  - しかし、10進数への変換は面倒
- ⇒16進数表記がよく用いられる
- 2進数⇔16進数
    - 2進数の4桁⇔16進数の1桁
    - 10101110⇔1010 1110⇔AE

| 2進   | 10進 | 16進 |
|------|-----|-----|
| 1001 | 9   | 9   |
| 1010 | 10  | A   |
| 1011 | 11  | B   |
| 1100 | 12  | C   |
| 1101 | 13  | D   |
| 1110 | 14  | E   |
| 1111 | 15  | F   |

## はじめに

- プロセッサの仕組みを理解するために**最小限、かつ限定的な説明**であり、**厳密ではない**部分がある。
  - 1Byteが8bitとは限らない
  - char型は8bitとは限らない
  - int型は16bitや32bit、64bitでも（18bit等でも）よい
  - char型が符号ありか符号なしかは実装依存⇒この講義では、大多数のコンピュータやコンパイラのルールに従う
- コードを読むための知識に限定しているため、正確なコード、あるいはそのままコンパイル可能なコードとは限らない

## 例題1

- 問題
  - 19を2の補数表現で表せ。ただし、語長は8bitとする。
- 解答

## 符号拡張

- 8bitのデータを16bitのデータに変換する
  - C/Java

```
char c; /* 8bit 符号付き整数 */
short s; /* 16bit 符号付き整数 */
c=10; /* cに10を代入 */
s=c; /* sにcの値を代入 */
```
- 符号なし(unsigned)の場合のビット拡張
  - 先頭を0で埋める
    - 00001010 ⇒ 0000000000001010
- 符号あり(signed)の場合のビット拡張
  - MSBの値で上位ビットを埋める
    - 00101111 ⇒ 0000000000101111
    - 10101111 ⇒ 1111111110101100

## コンピュータアーキテクチャを理解するための最小限のC言語

- 教科書はCの基礎ができている人向け
- プログラムを理解していないとハードウェアも理解できない
  - ハードウェアはプログラムを動かすもの
  - ハードウェアとソフトウェアの繋がりを理解

## C言語

- コンピュータ・アーキテクチャを学習する上でプログラムとの関係性は重要
- 様々な言語があるが、C言語は高級言語でありながら低レベルな記述（CPUに近いレベルで制御）ができる。
- アーキテクチャを説明する上で必要最小限のC言語の講義を行う（とりあえず書けなくても、読めればよい）。

## プログラム

- コンピュータに対する命令書
- 作業の手順を列挙したもの
- そのままではCPU上で実行できないので、コンパイル（翻訳）して機械語（CPUが理解できる形式）に変換して実行する。
- 主な構成要素
  - 変数
  - 文
  - 関数

## 配列

- 配列とは？
  - 複数の値を持つ変数
  - ベクトルや行列のようなもの  $1 \times N$ 、 $M \times N$ 、3次元以上でも可能
- 配列変数の宣言、アクセス

```
int i, vec[4], mat[2][3];
vec[0] = 10;
mat[1][2] = 3;
i = mat[0][0];
```

|        |        |        |        |
|--------|--------|--------|--------|
| vec[0] | vec[1] | vec[2] | vec[3] |
|--------|--------|--------|--------|

|           |           |           |
|-----------|-----------|-----------|
| mat[0][0] | mat[0][1] | mat[0][2] |
| mat[1][0] | mat[1][1] | mat[1][2] |

## 演算子（抜粋）

| 演算子          | 意味                                                                                |
|--------------|-----------------------------------------------------------------------------------|
| +, -, *, /   | 加減乗除                                                                              |
| %            | 剰余                                                                                |
| &,  , ^      | ビット毎の積、和、排他的論理和(XOR)                                                              |
| <<, >>       | シフト演算<br>$4 \ll 1 \Rightarrow 8$ ( $0100_2 \ll 1 \Rightarrow 1000_2$ )            |
| ==           | 比較演算：左右が同じ値なら1、違う値なら0                                                             |
| !=           | 比較演算：左右が同じ値なら0、違う値なら1                                                             |
| >, <, >=, <= | 比較演算：>=は $\geq$ （以上）, <=は $\leq$ （以下）                                             |
| !            | 否定演算（右側の値の否定を取る）<br>$!1 \Rightarrow 0$ , $!0 \Rightarrow 1$ , $!10 \Rightarrow 0$ |

## 制御構文(2/2)

- while文（繰り返し）

```
int i=0;
while(i < 10){
    i = i + 1;
}
```

  - iが10より小さい間はループ内の文を実行
  - iが10以上になるとループを抜ける

## 変数

- 変数は予め宣言する必要がある。
    - char型
      - 符号付き8bit
    - short型
      - 符号付き16bit
    - int/long型
      - 符号付き32bit
- ⇒型名の前にunsignedを付けると符号なしになる  
int a, b, c; /\* 32bitの変数 a, b, c を宣言 \*/

## 文

- 命令
    - 制御構文（後述）
    - 式
  - 式
    - a = 10;
    - a = 10 \* 20;
    - x = r \* sin(d\*3.14/180);
    - printf("hello world");
- =は代入（数学の等号とは少し意味が違う）  
a = 1; /\* aに1を代入 \*/  
a = a + 1; /\* aにa+1の結果を代入 \*/

## 制御構文(1/2)

- if文（条件実行）

```
if(a >= 10){
    aが10以上なら実行する文;
}
else {
    aが10未満なら実行する文;
}
```
  - for文（繰り返し）

```
for(i=0; i<10; i++){
    文; /* 10回実行 */
}
```
- } なしでもOK

## 関数

- 一連の手続きを定義したもの
- 数学の関数に似ているが、より柔軟（曖昧）なもの
  - t = sin(30 \* 3.14/180);
  - draw\_line(10, 20, 40, 50);
- 関数定義

```
関数の戻り値型 関数名(引数){
    変数宣言
    文
}
```

## 関数定義と関数呼び出し例1

- 整数を二つ受取って和を返す関数

```
int add(int a, int b){  
    return a + b;  
}
```

関数の呼び出し元に戻る

- add()関数を使って10+20を計算する

```
int t;  
t = add(10, 20);
```

- 例)

```
int is_larger(int a, int b){  
    if(a > b){  
        return 1;  
        a = 0;  
    }  
    return 0;  
}
```

a > bなら1を返し、  
これ以降の命令は  
実行しない。

この命令が実行さ  
れることはない！

## 関数定義と関数呼び出し例2

- 整数(N)を一つ受取って、1からNまでの総和を返す関数

```
int sum(int N){  
    int i, sum=0;  
    for(i = 1; i <= N; i = i + 1){  
        sum = sum + i;  
    }  
    return sum;  
}
```

- sum()関数を使って1~5までの数の総和を計算する

```
int w;  
w = sum(5);
```

## 変数のスコープ

- 関数の外で定義された変数（グローバル変数）は、どこからでもアクセスできる
- 関数の中で定義された変数（ローカル変数）は関数の中でしかアクセスできない

```
int x = 0;  
increment(int i){  
    int y = 0;  
    x = x + i;  
    y = y + i;  
    printf("%d %d\n", x, y);  
}
```

xの有効範囲

yの有効範囲

別の関数からも  
xは読み書き可

| 呼び出し回数 | x | y |
|--------|---|---|
| 1      | 1 | 1 |
| 2      | 2 | 1 |
| 3      | 3 | 1 |
| 4      | 4 | 1 |
| 5      | 5 | 1 |

## 再帰呼出し

- 関数の中から自分自身の関数を呼出す
- 1-Nの総和を求める関数を考える
  - 普通はループ文を使うが、あえて再帰呼出しを使う

```
int sum(int N){  
    if(N == 1){  
        return 1;  
    }  
    return N + sum(N-1);  
}
```

```
sum(1):  
return 1;
```

```
sum(2):  
return 2 + sum(1);  
sum(1):  
return 1;
```

```
sum(3):  
return 3 + sum(2);  
sum(2):  
return 2 + sum(1);  
sum(1):  
return 1;
```